

Actividad 5

Alumno

Alan Lam Páez

Matrícula

T03063809

Materia

Aplicaciones Interactivas

Maestro(a)

Cristopher Gerardo Gaytán Díaz

Correo

al03063809@tecmilenio.mx

Fecha

30/09/2026

Índice

Introducción	4
Caso: desarrollo conceptual de Adventure Campus	4
Fundamentos de Objetos 2D	5
I. ¿Qué es un sprite y cuál es su función en un videojuego 2D?	5
II. Textura vs. Sprite	5
III. ¿Qué es Paper2D y para qué se utiliza en Unreal Engine?	6
IV. ¿Qué es un Actor y cómo se relaciona con un componente PaperSprite?	6
V. La importancia de organizar los recursos gráficos en carpetas	7
VI. Cómo transformar una imagen en un objeto utilizable dentro de un videojuego (Paper 2D)	8
Análisis del rol de una cámara en un videojuego 2D	11
I. ¿Qué es una cámara y por qué es crítica en un juego 2D?	11
II. Camera Component vs. Camera Actor: el caso de Celeste	11
III. Proyección ortográfica y su impacto en la jugabilidad	12
IV. Location, Rotation y Orthographic Width en la práctica	12
V. ¿Qué significa que la cámara siga al personaje?	12
VI. Problemas de una cámara mal configurada	13
VII. Más Acerca de los Tipos de Cámaras 2D	14
Física y Colisiones	15
I. Conceptos fundamentales: simulación física y detección de colisiones (Epic Games, 2025)	15
II. Block, Overlap e Ignore	16
III. Collision Presets	16
IV. Estático vs. con simulación física	17
V. Gravedad y fuerzas	17
VI. Colisiones mal configuradas (problemas comunes)	18
VI.. Ejemplos de Configuraciones de Colisión para Adventure Campus	19
Análisis de la lógica de interacción	21
Escenario 1: El personaje cae y entra en contacto con una plataforma	21

Pseudocódigo:	22
Escenario 2: El personaje toca un objeto recolectable	22
Escenario 3: La cámara sigue al personaje durante su movimiento	23
Análisis de un videojuego 2D: Celeste	25
Análisis Lógico: “¿Qué debería suceder?”	27
Escenario 1. Personaje y plataforma	27
Escenario 2. Objeto recolectable	28
Conclusiones	29
Bibliografía	30

Introducción

El presente documento analiza los fundamentos de los sprites y Paper2D, el papel de la cámara en la percepción del jugador, la simulación física y la detección de colisiones, así como la lógica condicional que permite controlar las interacciones entre actores. A partir del caso conceptual *Adventure Campus* y tomando como referente el diseño de *Celeste*, se busca sentar las bases teóricas y prácticas necesarias antes de iniciar la implementación en Unreal Engine.

Caso: desarrollo conceptual de Adventure Campus

Una empresa de videojuegos está diseñando un videojuego de plataformas 2D llamado Adventure Campus, donde el jugador controla un personaje que debe recorrer diferentes escenarios, evitar obstáculos y recolectar objetos.

El equipo de desarrollo necesita definir cómo funcionarán los objetos del escenario antes de iniciar la implementación en Unreal Engine.

Durante la fase de análisis, se identifican los siguientes problemas:

- El personaje debe visualizarse correctamente en el escenario.
- La cámara debe seguir al jugador sin perder de vista los elementos importantes.
- El personaje debe caer cuando no exista una plataforma debajo.
- Las plataformas deben impedir que el personaje atraviese su superficie.
- Los objetos recolectables deben detectar el contacto con el jugador sin bloquear su movimiento.
- Algunos objetos deben permanecer estáticos, mientras que otros deben reaccionar a fuerzas físicas.

La finalidad del documento es analizar estos problemas y explicar qué conceptos, componentes y reglas lógicas podrían utilizarse para resolverlos.

Fundamentos de Objetos 2D

I. ¿Qué es un sprite y cuál es su función en un videojuego 2D?

En esencia, un **sprite** es la representación visual de una entidad (o actor) dentro del juego. Se integra dentro de una escena más grande, generalmente en un videojuego 2D, como un tipo de mapa de bits que se dibuja en la pantalla para representar elementos gráficos como personajes, objetos, enemigos o elementos del entorno.

Los sprites suelen ser pequeños y pueden incluir zonas de transparencia, lo que permite que no se rendericen como rectángulos rígidos. Además, se pueden animar como en los dibujos animados: mediante la secuencia de varios sprites que producen la ilusión de movimiento de un personaje, como correr, saltar o recibir daño (Hobby Consolas, 2021).

En el contexto del desarrollo con Unreal Engine, un sprite se define como una textura bidimensional estática que se utiliza para representar personajes u objetos, y suele organizarse en hojas de sprites (*sprite sheets*) que contienen múltiples variaciones para animación (Epic Games, 2025).

II. Textura vs. Sprite

Aunque en la práctica ambos términos suelen confundirse, existe una diferencia conceptual:

Concepto	Definición	Función principal
Textura	Es una imagen cargada en la memoria de la tarjeta gráfica. Se utiliza para cambiar la apariencia de un objeto, ya sea en 2D o en 3D.	Proporciona el aspecto visual de una superficie o modelo al aplicarse con un <i>Material</i> . Puede aplicarse sobre geometría 3D o servir como base para un sprite.
Sprite	Es un objeto 2D que utiliza una textura para mostrarse en pantalla, pero que además posee propiedades propias como posición (x, y), rotación y escala,	Representa una entidad visual o asset con comportamiento dentro del juego 2D. Es un "lienzo" que muestra una

	y puede ser movido, creado o destruido durante el juego.	textura y sobre el que se puede operar.
--	--	---

III. ¿Qué es Paper2D y para qué se utiliza en Unreal Engine?

Según Epic Games (2025), Paper2D es un sistema basado en sprites que forma parte de Unreal Engine y que está diseñado específicamente para la creación de juegos 2D o juegos híbridos 2D/3D. Se distribuye como un plugin que debe activarse desde el menú de plugins del editor.

Sus usos principales incluyen:

- **Creación de personajes y objetos 2D:** Permite importar imágenes, convertirlas en sprites y utilizarlas para representar entidades del juego.
- **Animación mediante Flipbooks:** Los flipbooks son secuencias de sprites que se reproducen de forma ordenada para crear animaciones, como caminar o atacar
- **Construcción de niveles 2D:** A través de TileSets y TileMaps, se pueden crear fondos y entornos a partir de conjuntos de texturas repetibles.
- **Integración con la lógica de Unreal:** Los sprites y flipbooks se integran como componentes dentro de Actors y Blueprints, lo que permite combinarlos con la física, las colisiones y la programación visual del motor.

IV. ¿Qué es un Actor y cómo se relaciona con un componente PaperSprite?

Un **Actor** en Unreal Engine es la unidad fundamental de cualquier objeto que puede colocarse en un nivel. Prácticamente todo lo que existe en un escenario de Unreal (personajes, plataformas, ítems, luces, cámaras) es un Actor o deriva de él. Un Actor puede estar vacío o contener múltiples componentes que definen su apariencia, comportamiento y propiedades físicas.

El **PaperSpriteComponent** es un tipo de componente específico de Paper2D que se encarga del renderizado y la colisión de una instancia de un asset UPaperSprite. La relación entre Actor y PaperSpriteComponent funciona de la siguiente manera:

1. Cuando se arrastra un **sprite asset** desde el Content Browser hacia un Blueprint o directamente a un nivel, Unreal Engine crea automáticamente un Actor (o lo modifica) que contiene un **PaperSpriteComponent**.

2. Este componente es el que muestra visualmente el sprite en la escena y, si se configura, también gestiona su colisión (por ejemplo, para que una plataforma impida el paso del jugador).
3. Si se arrastra un sprite directamente a un nivel, Unreal genera un **PaperSpriteActor**, que es un Actor predefinido con un PaperSpriteComponent instanciado.

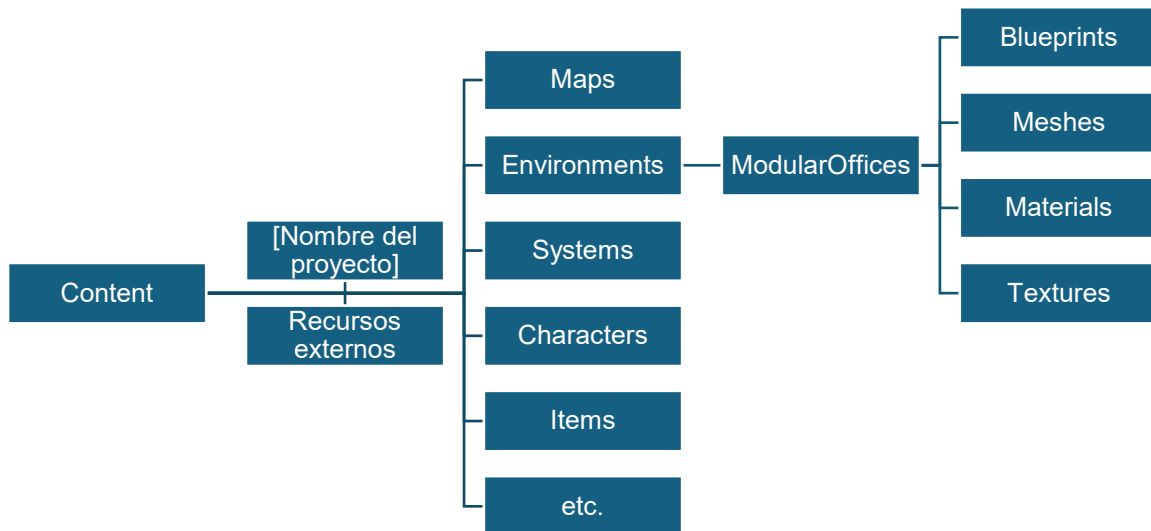
V. La importancia de organizar los recursos gráficos en carpetas

En Unreal Engine, la organización de los recursos (como texturas, sprites, materiales, animaciones, etc.) en una estructura de carpetas coherente es importante, especialmente en proyectos que crecen en tamaño y complejidad.

Algunas ventajas de mantener una organización coherente incluyen (Strayspark Studio, 2026):

Ventaja	Descripción
Eficiencia en la búsqueda de assets	Un proyecto con una estructura clara permite localizar rápidamente cualquier recurso sin perder tiempo navegando por directorios desordenados.
Facilitación del trabajo en equipo	Diferentes roles (artistas, programadores, diseñadores) pueden trabajar en carpetas separadas según su área, lo que evita conflictos y facilita la colaboración.
Mantenimiento de referencias	Mover assets de forma desorganizada puede romper las referencias dentro de Unreal Engine. Una estructura estable desde el inicio del proyecto evita este problema.
Optimización de builds	Una organización adecuada permite excluir carpetas de desarrollo o pruebas de las compilaciones finales, reduciendo el tamaño y los tiempos de construcción.

a. *Ejemplo de una estructura ordenada*



Idealmente, los recursos deberían alojarse en una carpeta adicional en Content, no dentro del mismo Content. Ya que las descargas de Fab (desde el plugin en Unreal Engine) se alojan automáticamente en Content, sin la posibilidad de elegir otra ruta. Además, esta estructura evita que en el futuro las carpetas importantes del proyecto se mezclen con otras carpetas arbitrarias.

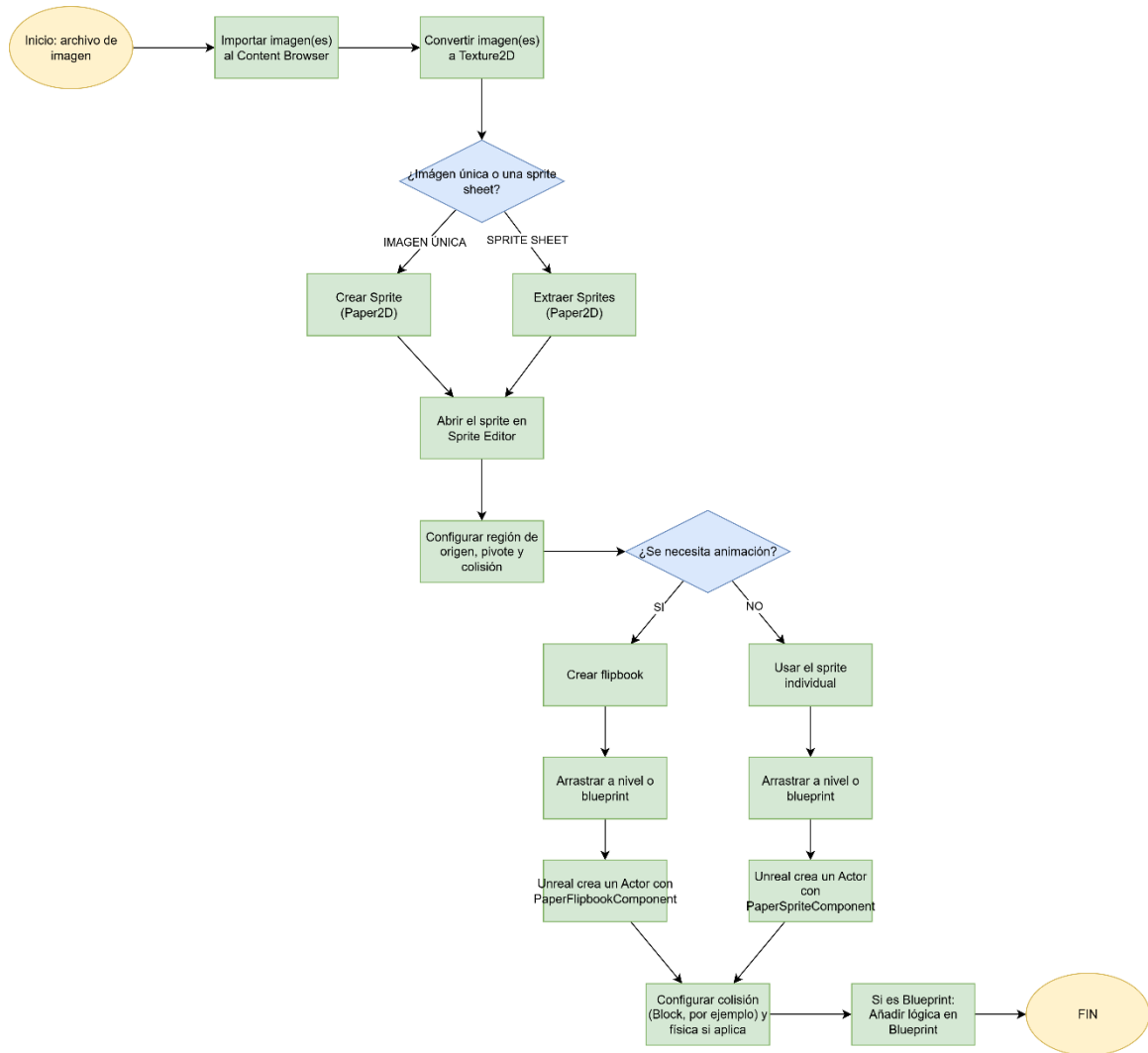
VI. **Cómo transformar una imagen en un objeto utilizable dentro de un videojuego (Paper 2D)**

El proceso para convertir una imagen, por ejemplo, un archivo PNG, en un objeto utilizable dentro de Unreal Engine con Paper2D sigue una serie de pasos concretos:

1. **Importar la imagen como textura:** Se arrastra el archivo de imagen al Content Browser o se utiliza la opción de importar. En este punto, el recurso existe como una textura (Texture2D), pero aún no puede utilizarse como objeto de juego (Peake, 2016).
2. **Crear el sprite a partir de la textura:** Se hace clic derecho sobre la textura importada y se selecciona la opción Paper2D / Create Sprite (o "Create Sprite" en versiones más recientes). Esto genera un nuevo asset de tipo Sprite que hace referencia a la textura original (Peake, 2016).

3. **Configurar el sprite en el Sprite Editor:** Se abre el sprite recién creado en el editor. Ahí se puede ajustar la región de origen (si la imagen contiene varios sprites), el pivote (punto de referencia para rotación y posicionamiento) y las propiedades de colisión (Peake, 2016).
 - a) **Extraer múltiples sprites de una hoja de sprites (opcional):** Si la imagen es una *sprite sheet* con varios fotogramas de animación, se puede hacer clic derecho sobre la textura y usar **Sprite Actions / Extract Sprites**. Esto permite extraer automáticamente cada fotograma como un sprite independiente, ajustando la cuadrícula de extracción (Peake, 2016).
 - b) **Crear un Flipbook para animación (opcional):** Una vez extraídos los sprites individuales, se pueden seleccionar todos y hacer clic derecho para **Create Flipbook**, lo que genera un asset de animación que reproduce la secuencia de sprites (Peake, 2016).
4. **Integrar el sprite en un Actor o Blueprint:** Finalmente, el sprite se arrastra a un Blueprint o a un nivel, lo que genera un **PaperSpriteComponent** dentro de un Actor. Desde ahí, el sprite ya forma parte del juego y puede recibir lógica, colisiones y comportamiento (Epic Games, 2014).

a. Ejemplo visual de la implementación de una imagen (Textura) a un Sprite utilizable en un nivel o blueprint



Análisis del rol de una cámara en un videojuego 2D

- **Videojuego 2D de referencia:** Celeste (2018)

Celeste es un videojuego de plataformas 2D de precisión desarrollado por Maddy Makes Games y lanzado en 2018. La historia sigue a Madeline, una joven que escala la montaña Celeste mientras enfrenta sus propios demonios internos. El juego combina una dificultad exigente con un diseño de niveles meticuloso, donde el jugador domina un set limitado de movimientos: correr, saltar, escalar y dash, en pantallas que funcionan como pequeños rompecabezas de precisión. Su reaparición instantánea invita a reintentar sin castigo, y la narrativa vincula la jugabilidad con temas de superación personal y salud mental.

I. ¿Qué es una cámara y por qué es crítica en un juego 2D?

La cámara es el punto de vista del jugador: define qué se ve, desde dónde y cómo. En Unreal Engine, el `PlayerController` instancia un `Camera Actor` que calcula la posición y orientación desde la que se observa el mundo (Epic Games, 2021). En un juego 2D, la cámara no puede rotar libremente como en 3D. Su función principal es **enmarcar la acción** mediante desplazamientos sobre un plano (scrolling).

En *Celeste*, la cámara es ortográfica, con una escala constante que permite medir distancias al píxel. El jugador nunca duda de dónde está una plataforma o a qué distancia está un peligro.

II. Camera Component vs. Camera Actor: el caso de Celeste

En Unreal Engine, el **Camera Component** contiene las propiedades de la cámara (proyección, FOV, Ortho Width, postprocesado), mientras que el **Camera Actor** es la instancia que permite colocarla en el nivel (Epic Games, 2021). Es decir, el componente define *cómo* funciona la cámara; el Actor, *dónde* está.

En *Celeste*, se podría imaginar esta estructura como un `Camera Actor` que sigue a Madeline, con un `Camera Component` configurado en modo ortográfico. La cámara de *Celeste*, sin embargo, no está pegada al personaje: tiene amortiguación (damping), zona muerta horizontal y un eje vertical conservador que ignora saltos pequeños. Esa lógica de seguimiento brinda personalidad a un `Camera actor` en contraste con otros juegos de plataformas.

III. Proyección ortográfica y su impacto en la jugabilidad

La proyección ortográfica elimina la distorsión de profundidad: los objetos mantienen su tamaño sin importar la distancia (Epic Games, 2024). En 2D, esto es esencial porque:

- Permite calcular saltos con precisión.
- Garantiza que lo que se ve en el editor es lo que ve el jugador, sin situaciones inesperadas de perspectiva.
- Facilita la lectura visual de plataformas, obstáculos y coleccionables.

Si Celeste usara perspectiva, los saltos de precisión serían injustos, ya que el jugador no podría confiar en sus ojos para medir distancias.

IV. Location, Rotation y Orthographic Width en la práctica

Estos tres parámetros son los controles fundamentales de una cámara 2D:

Parámetro	Qué controla	En <i>Celeste</i>
Location	Posición en el mundo	Se actualiza para seguir a Madeline con damping
Rotation	Orientación de la vista	Fija: mira de frente al plano de juego
Orthographic Width	Nivel de zoom (ancho visible)	Constante durante el gameplay; se ajusta en momentos narrativos

En *Celeste*, el Ortho Width se mantiene estable durante el juego para no romper la legibilidad espacial. Solo en secuencias específicas (como la revelación de un desafío o escenas emocionales) la cámara se aleja o acerca sutilmente, usando el zoom como recurso expresivo sin afectar la jugabilidad.

V. ¿Qué significa que la cámara siga al personaje?

Seguir al personaje no es literalmente pegar la cámara encima. Si la cámara se moviera exactamente igual que Madeline, cada salto provocaría un movimiento vertical brusco y cada pequeño ajuste sacudiría la pantalla. Celeste evita esto con varias técnicas:

- **Damping suave:** la cámara se desplaza hacia su objetivo con una demora gradual, no instantánea (Brown, 2023).
- **Zona muerta horizontal:** Madeline puede moverse ligeramente sin que la cámara reaccione, permitiendo ajustes precisos sin sacudidas.
- **Eje vertical conservador:** la cámara ignora saltos pequeños y solo se desplaza verticalmente cuando Madeline cambia de zona claramente.
- **Lookahead:** la cámara se adelanta ligeramente en la dirección del movimiento, revelando obstáculos antes de que el jugador llegue a ellos (Brown, 2023).

VI. Problemas de una cámara mal configurada

Problema	Consecuencia	Ejemplo de cómo lo evita Celeste
Personaje sale de pantalla	Pérdida de control visual	Seguimiento con damping y zona muerta
Movimiento vertical brusco	Mareo, incomodidad	Eje vertical conservador que ignora saltos
Sacudida (jitter)	Sensación de falta de pulido	Suavizado constante en todos los ejes
Efecto túnel	Muertes por peligros fuera de pantalla	Lookahead que revela obstáculos próximos
Pixel shimmer	Sprites borrosos o vibrantes	Alineación a la cuadrícula de píxeles
Pérdida de referencia espacial	Incapacidad de planificar rutas	Escala constante y movimiento predecible

Brown (2023) señala que una cámara 2D debe priorizar la visibilidad del jugador y anticipar sus necesidades. Celeste lo logra porque el jugador nunca muere por algo que no vio, y cuando muere, sabe exactamente por qué.

VII. Más Acerca de los Tipos de Cámaras 2D

Tipo de cámara	Proyección	Uso típico en 2D	Características principales	Ejemplo de juego
Cámara lateral fija (Side-on fija)	Ortográfica	Juegos de plataformas, acción lateral	La cámara no se mueve; el personaje se desplaza dentro de un encuadre fijo	<i>Super Mario Bros.</i> (niveles con scroll bloqueado)
Cámara lateral con scroll (Side-scrolling)	Ortográfica	Plataformas, shoot 'em up, aventuras	La cámara sigue al personaje horizontalmente, a menudo con amortiguación	<i>Celeste</i> , <i>Hollow Knight</i>
Cámara cenital (Top-down)	Ortográfica	RPG, estrategia, aventuras	Vista desde arriba; la cámara sigue al personaje en ambos ejes (X e Y)	<i>The Legend of Zelda</i> (vista cenital), <i>Stardew Valley</i>
Cámara isométrica	Ortográfica (con ángulo)	Estrategia, RPG táctico, simulación	Vista en ángulo de 45°; simula profundidad sin usar perspectiva	<i>Diablo</i> , <i>Age of Empires</i>
Cámara 2.5D	Ortográfica o perspectiva	Plataformas modernas, acción	Gráficos 3D sobre jugabilidad 2D; la cámara puede tener ligera perspectiva	<i>Ori and the Blind Forest</i> , <i>Little Nightmares</i>
Cámara de pantalla dividida (Split-screen)	Ortográfica	Multijugador local	Múltiples cámaras que siguen a diferentes jugadores en la misma pantalla	<i>Cuphead</i> (modo cooper)

Física y Colisiones

I. Conceptos fundamentales: simulación física y detección de colisiones (Epic Games, 2025).

Según Epic Games (2025), la simulación física en Unreal Engine se encarga de calcular cómo se mueven los objetos en respuesta a fuerzas como la gravedad, el impulso o la fricción. Cuando un objeto tiene activada la opción *Simulate Physics*, el motor calcula su posición y rotación en cada fotograma aplicando las leyes de Newton: si está en el aire, cae; si recibe un empujón, se desplaza; si choca con algo, rebota o se detiene según su masa y elasticidad. En Unreal Engine, para juegos 2D laterales, es común aplicar **constraints** que bloquean el movimiento en el eje Y y la rotación en el eje X, de modo que el sprite no salga del plano de juego.

La **detección de colisiones**, en cambio, no calcula movimiento. Se limita a responder: "¿estos dos objetos se están tocando?" y a decidir qué ocurre cuando la respuesta es sí. Esta decisión se toma a través de **respuestas de colisión**, que en Unreal Engine son: *Block*, *Overlap* e *Ignore*. Un objeto con simulación física puede tener cualquier combinación de estas respuestas. Un objeto sin simulación (estático) también puede tenerlas, y de hecho las necesita para que la detección funcione. Una colisión puede no reaccionar físicamente a ella (*Overlap*), o puede bloquear el paso sin que eso active ningún evento visible en el juego (*Block* sin *Generate Hit Events*).

II. Block, Overlap e Ignore

Respuesta	¿Qué hace?	¿Genera evento?	Uso típico en 2D
Block	Impide que los objetos se atraviesen. La física se detiene en el punto de contacto.	Sí, si <i>Simulation Generates Hit Events</i> está activado.	Suelo, paredes, plataformas sólidas.
Overlap	Los objetos se atraviesan, pero se detecta el contacto y se puede reaccionar a él.	Sí, mediante <i>Generate Overlap Events</i> .	Monedas, checkpoints, zonas de daño, triggers.
Ignore	No ocurre nada. Es como si los objetos existieran en mundos separados.	No.	Fondos decorativos, capas de parallax, objetos que no deben interactuar.

Epic Games (2025); Unreal Engine (2014).

III. Collision Presets

Un **Collision Preset** es una plantilla que agrupa un conjunto de respuestas predefinidas para todas las categorías de objetos del motor (Pawn, WorldStatic, WorldDynamic, PhysicsBody, etc.). En lugar de configurar manualmente cada respuesta una por una, se selecciona un preset que ya define el comportamiento típico de ese tipo de objeto (Epic Games, 2025).

Los presets más comunes en proyectos 2D son:

- **BlockAll**: bloquea absolutamente todo.
- **OverlapAll**: detecta todo pero no bloquea nada.
- **Pawn**: preset estándar para personajes controlados. Bloquea geometría y detecta overlaps con triggers.
- **PhysicsActor**: para objetos que simulan física. Bloquea geometría sólida.

IV. Estático vs. con simulación física

Un objeto **estático** (sin *Simulate Physics*) permanece donde se coloca. Su posición no cambia a menos que el código lo mueva explícitamente. En un juego 2D, esto aplica a plataformas fijas, paredes, suelos y fondos. La detección de colisiones sigue funcionando: el personaje puede bloquearse contra ellos o detectar overlaps.

Un objeto **con simulación física** responde a fuerzas. Si tiene gravedad, cae; si recibe un impulso, se mueve; si choca, reacciona según su masa. En un juego 2D, esto se usa para objetos como cajas que caen, plataformas móviles empujadas por el jugador, o proyectiles afectados por gravedad. Sin constraints, un sprite con física puede rotar fuera del plano 2D y "desaparecer" del encuadre; por eso se bloquean los ejes no deseados.

Entonces, **estático es predecible, y físico es reactivo pero menos controlable**. En Adventure Campus, las plataformas principales deberían ser estáticas. Los objetos recolectables que caen o se deslizan podrían tener simulación física con constraints.

V. Gravedad y fuerzas

En Unreal Engine, la gravedad por defecto es de -980 cm/s^2 en el eje Z (Epic Games, 2025). Cuando un objeto tiene *Simulate Physics* activado, el motor aplica esta aceleración en cada fotograma, y la velocidad del objeto aumenta progresivamente hasta que algo lo detiene (una colisión con Block, por ejemplo).

Las fuerzas son empujones puntuales o continuos que se aplican a un objeto físico. En el contexto de un juego de plataformas 2D, las fuerzas se usan para:

- **Salto:** se aplica un impulso vertical al personaje cuando el jugador presiona el botón.
- **Dash o impulso horizontal:** se aplica una fuerza en X durante un tiempo determinado.
- **Rebotes:** al colisionar con un objeto elástico, se invierte parte de la velocidad.
- **Plataformas móviles:** se mueven con una fuerza constante o con interpolación, y el personaje hereda su momentum al saltar (como en *Celeste*).

La gravedad y las fuerzas no son exclusivas de la simulación física completa. Un personaje controlado por Character Movement Component también puede recibir gravedad y fuerzas, pero el motor gestiona su movimiento de forma más controlada que un objeto puramente físico (Epic Games, 2025).

VI. Colisiones mal configuradas (problemas comunes)

Problema	Causa	Consecuencia
El personaje atraviesa el suelo	El suelo no está bloqueando al Pawn, o el Pawn no está bloqueando al suelo.	Caída infinita; el jugador no puede progresar.
Los coleccionables bloquean el paso	El objeto recolectable tiene respuesta Block en lugar de Overlap.	El jugador se detiene al tocarlos, en lugar de recogerlos.
Los triggers no se activan	Falta <i>Generate Overlap Events</i> , o los objetos están en canales que se ignoran mutuamente.	Las puertas no se abren, los checkpoints no se registran.
Objetos físicos salen del mundo	Sin constraints, el sprite rota o se desplaza en ejes no deseados.	El objeto cae fuera del nivel o se vuelve invisible.
Colisiones fantasma	Dos objetos con Block se solapan porque uno fue teletransportado sin actualizar la física.	El jugador queda atrapado o es expulsado.
Overlaps repetidos	El evento Overlap se dispara múltiples veces por fotograma.	El contador de monedas aumenta de más.

Fuente: Epic Games (2025); Unreal Engine (2014).

VI.. Ejemplos de Configuraciones de Colisión para Adventure Campus

Configuración	Respuesta	Preset sugerido	Simulación física	Aplicación en Adventure Campus
Plataforma sólida	Block (Pawn)	BlockAll	No	Suelo, paredes y plataformas fijas. El personaje no puede atravesarlas.
Moneda recolectable	Overlap (Pawn)	OverlapAllDynamic	No	Al tocar la moneda, se detecta el contacto y se suma al contador sin detener al personaje.
Zona de daño (pinchos)	Overlap (Pawn)	OverlapAllDynamic	No	Detecta que el personaje entró en la zona y aplica daño o muerte, sin bloquear su movimiento.
Caja con física	Block (Pawn y WorldStatic)	PhysicsActor	Sí (con constraints)	Caja que cae y puede ser empujada por el personaje. Bloquea el paso pero reacciona a la gravedad.
Plataforma móvil	Block (Pawn)	BlockAllDynamic	No (movida por código)	Se mueve de forma interpolada; el personaje "hereda" su momentum al saltar desde ella.

Trigger de checkpoint	Overlap (Pawn)	OverlapOnlyPawn	No	Al entrar en la zona, se guarda el progreso. No bloquea ni afecta la física.
------------------------------	----------------	-----------------	----	--

a) Ejemplo de aplicación (Celeste)

En el caso de Celeste, no se utiliza un componente de física incluido por defecto en un motor de juego. Maddy Thorson, su creadora, desarrolló un sistema propio con dos clases: **Solids** (geometría de nivel: suelo, paredes, plataformas) y **Actors** (objetos físicos: jugador, enemigos, proyectiles) (Thorson, s.f.).

Nunca hay overlap: En *Celeste*, los Solids y los Actors **nunca se superponen**, excepto en circunstancias especiales. Todos los colisionadores son **AABBs** (cajas alineadas a los ejes) con posiciones enteras, lo que garantiza precisión al píxel (Thorson, s.f.). Cuando un Actor intenta moverse hacia una posición ocupada por un Solid, el sistema mueve al Actor píxel a píxel hasta que la colisión se detecta, y entonces ejecuta una función de callback que define qué ocurre (rebote, detención, muerte, etc.).

Este enfoque es distinto al de Unreal Engine. En Unreal, el motor de física gestiona las colisiones de forma automática. En *Celeste*, el desarrollador escribe explícitamente cada comprobación. La ventaja principal es un control absoluto sobre el comportamiento.

¿Como se ajusta a Adventure Campus? Para Adventure Campus, no es necesario replicar el sistema de *Celeste* desde cero, pero se podrían adoptar sus principios:

1. **Precisión al píxel:** aunque Unreal trabaje con floats, configurar el personaje y las plataformas en una cuadrícula coherente reduce la imprevisibilidad.
2. **Respuestas explícitas:** cada tipo de colisión debe tener una respuesta clara: Block para lo sólido, Overlap para lo detectable, Ignore para lo decorativo.
3. **Sin overlaps accidentales:** si el personaje no debe atravesar una plataforma, ambos deben estar configurados para bloquearse mutuamente. Un error en el preset (por ejemplo, dejar el suelo en OverlapAll) rompe el juego.

4. **Constraints en objetos físicos:** cualquier objeto con simulación física debe tener bloqueados los ejes que no corresponden al plano 2D, como hace *Ce/este* al restringir todo a un plano de píxeles.

Análisis de la lógica de interacción

En Unreal Engine, las reglas de un videojuego se representan mediante **lógica condicional** implementada en Blueprints (programación visual) o en C++. Cada interacción sigue una estructura común: un **evento** dispara la evaluación de una **condición**, y si esta se cumple, se ejecuta una **acción** que produce un **resultado** observable en el juego. A continuación, se desarrollan tres escenarios aplicados a Adventure Campus, con su pseudocódigo correspondiente.

Escenario 1: El personaje cae y entra en contacto con una plataforma

Elemento	Descripción
Evento	Tick (cada fotograma) o evento de colisión del componente de movimiento.
Objetos involucrados	Personaje (Pawn con Character Movement), Plataforma (Actor estático con colisión Block).
Condición	La velocidad vertical del personaje es negativa (está cayendo) y su hitbox inferior intersecta la hitbox superior de la plataforma.
Acción	Detener la caída, ajustar la posición Z del personaje al borde superior de la plataforma, resetear velocidad vertical a cero, marcar estado "en suelo".
Resultado	El personaje queda de pie sobre la plataforma y recupera la capacidad de saltar.

Pseudocódigo:

```
EVENTO: Tick

  SI personaje.velocidadZ < 0
    Y personaje.hitboxInferior intersecta plataforma.hitboxSuperior:
      personaje.posicionZ = plataforma.posicionZ + plataforma.altura
      personaje.velocidadZ = 0
      personaje.estaEnSuelo = verdadero
  FIN SI
FIN EVENTO
```

Explicación: La condición clave es que el personaje esté cayendo (velocidadZ < 0) y que su parte inferior toque la parte superior de la plataforma. Sin esa doble condición, el personaje podría "pegarse" a la plataforma desde abajo o lateralmente.

Escenario 2: El personaje toca un objeto recolectable

Elemento	Descripción
Evento	OnOverlap (evento de superposición) entre el hitbox del personaje y el hitbox de la moneda.
Objetos involucrados	Personaje (Pawn con colisión Block/Overlap), Moneda (Actor con colisión Overlap).
Condición	La moneda está activa (moneda.activa == verdadero).
Acción	Incrementar el contador de monedas, reproducir sonido y efecto visual, desactivar la moneda (visible = falso, colisión = Ignore).
Resultado	El contador sube, la moneda desaparece de la pantalla y el jugador recibe retroalimentación sonora y visual.

Pseudocódigo:

```
EVENTO: OnOverlap(personaje, moneda)

    SI moneda.activa == verdadero:

        contadorMonedas = contadorMonedas + 1

        reproducirSonido("moneda")

        moneda.visible = falso

        moneda.colision = Ignore

        moneda.activa = falso

    FIN SI

FIN EVENTO
```

Explicación: La moneda debe estar configurada como Overlap, no como Block. Si estuviera en Block, el personaje se detendría al tocarla. La condición moneda.activa evita que el contador sume más de una vez si el evento Overlap se dispara en varios fotogramas.

Escenario 3: La cámara sigue al personaje durante su movimiento

Elemento	Descripción
Evento	Tick (cada fotograma).
Objetos involucrados	Cámara (Camera Actor con proyección ortográfica), Personaje (Pawn).
Condición	La posición del personaje está fuera de la zona muerta horizontal de la cámara, o fuera del umbral vertical permitido.
Acción	Desplazar la cámara hacia la posición del personaje con interpolación (damping), aplicando lookahead horizontal y restringiendo el eje vertical.
Resultado	La cámara acompaña al personaje de forma suave, manteniéndolo visible sin movimientos bruscos.

Pseudocódigo:

```
EVENTO: Tick

// Eje horizontal con lookahead y zona muerta
objetivoX = personaje.posicionX + (personaje.velocidadX *
factorLookahead)

SI personaje.posicionX > camara.posicionX + zonaMuertaDerecha:
    camara.posicionX = interpolar(camara.posicionX, objetivoX,
dampingHorizontal)
FIN SI

SI personaje.posicionX < camara.posicionX - zonaMuertaIzquierda:
    camara.posicionX = interpolar(camara.posicionX, objetivoX,
dampingHorizontal)
FIN SI

// Eje vertical conservador (ignora saltos pequeños)
SI personaje.posicionZ > camara.umbralVerticalSuperior
    O personaje.posicionZ < camara.umbralVerticalInferior:
        camara.posicionZ = interpolar(camara.posicionZ,
personaje.posicionZ, dampingVertical)
FIN SI
FIN EVENTO
```

Explicación: La zona muerta permite que el personaje se mueva ligeramente sin que la cámara reaccione. El lookahead desplaza la cámara en la dirección del movimiento para revelar obstáculos próximos. El eje vertical solo se ajusta cuando el personaje sale de un rango predefinido, evitando que cada salto sacuda la pantalla.

Análisis de un videojuego 2D: Celeste

Nota: Este apartado retoma lo ya analizado sobre Celeste en secciones anteriores.

Aspecto	Análisis
Nombre y género	Celeste (2018), desarrollado por Maddy Makes Games. Género: plataformas 2D de precisión.
Perspectiva o cámara	Cámara lateral con proyección ortográfica. Sigue a Madeline con damping suave, zona muerta horizontal, lookahead y eje vertical conservador que ignora saltos pequeños.
Sprites y objetos visuales	Sprites en pixel art con escala constante. Cada entidad (Madeline, plataformas, pinchos, coleccionables) es un sprite con hitbox propia. Los recursos están organizados por tipo (personajes, entornos, efectos).
Física o movimiento	Gravedad aplicada al personaje, salto con impulso vertical, dash con fuerza horizontal, momentum heredado de plataformas móviles. El motor es propio, no comercial: los Solids y Actors se mueven píxel a píxel.
Colisiones	AABB alineadas a los ejes, posiciones enteras. Nunca hay overlap entre Solids y Actors: cuando un Actor intenta ocupar la posición de un Solid, se ejecuta un callback que define la respuesta (detención, rebote, muerte).
Interacciones jugador-entorno	Plataformas sólidas (Block), pinchos y zonas de daño (Overlap con reinicio), coleccionables (Overlap sin bloquear), objetos interactivos como resortes y bloques rompibles.

Problema potencial de diseño y propuesta conceptual

Problema: Celeste es un juego extremadamente difícil. La precisión al píxel y la ausencia de asistencia pueden generar frustración en jugadores noveles, especialmente en los capítulos avanzados (como el Capítulo 9, Farewell) donde

los inputs deben ser casi perfectos. La cámara, aunque bien diseñada, no ofrece ayuda adicional al jugador cuando este se queda atascado en una sección.

Propuesta conceptual: Implementar un **sistema de asistencia opcional** (como el Assist Mode que el juego ya incluye, pero llevado más lejos) que ajuste dinámicamente parámetros de física y cámara según el rendimiento del jugador. Por ejemplo:

- Si el jugador muere más de N veces en una misma pantalla, aumentar ligeramente la zona muerta de la cámara y reducir la velocidad de los objetos peligrosos.
- Ofrecer un modo de cámara ampliada (mayor Ortho Width) que muestre más entorno, facilitando la planificación de rutas.
- Reducir la gravedad efectiva en un porcentaje mínimo para dar más tiempo de reacción en saltos.

Lo importante es que la asistencia sea invisible y progresiva. El jugador no debería sentir que el juego "le está ayudando", sino que simplemente está mejorando. Esto preserva la sensación de logro que define a Celeste mientras reduce la barrera de entrada para nuevos jugadores.

Análisis Lógico: “¿Qué debería suceder?”

Escenario 1. Personaje y plataforma

Un personaje se encuentra en el aire y cae debido a la gravedad. Debajo existe una plataforma que debe impedir que el personaje la atraviese.

Analiza y responde:

- ¿Qué objetos intervienen?
- ¿Qué configuración de colisión debería tener la plataforma?
- ¿Qué comportamiento físico se espera del personaje?
- ¿Qué evento podría ocurrir cuando el personaje toca la plataforma?
- ¿Qué pasaría si la plataforma tuviera una configuración de colisión incorrecta?

Tomando como referencia un Third Person Template de Unreal Engine, cuando el personaje cae desde el aire, los objetos que intervienen son el Character (el personaje), el Character Movement Component asociado a él, y al menos un Static Mesh o un Blueprint con Static Mesh que actúe como plataforma y tenga colisiones habilitadas.

La plataforma no requiere una colisión de polígonos compleja; basta con que la forma de la colisión se ajuste lo más posible al Static Mesh visible. El contacto con el Character debe estar configurado como Block para impedir que el personaje la atraviese. Si no se bloquea al actor correspondiente, la colisión simplemente no se aplicará para él.

El comportamiento físico esperado del personaje es detener su caída al entrar en contacto con la superficie superior de la plataforma, reiniciar su velocidad vertical a cero y quedar en estado "en suelo", recuperando la capacidad de saltar.

El evento que podría ocurrir al tocar la plataforma es un OnComponentHit o una detección de suelo (landed) dentro del Movement Component, que puede disparar lógica adicional: reproducir un sonido de aterrizaje, activar una animación, o ejecutar un evento en Blueprint.

Si la plataforma tuviera una configuración incorrecta —por ejemplo, Overlap en lugar de Block, o sin colisiones habilitadas— el personaje la atravesaría y caería indefinidamente fuera del nivel. Si tuviera Block pero con una forma de colisión mal ajustada, el personaje podría quedar flotando en el aire o atravesar visualmente la plataforma antes de detenerse.

Adicionalmente, si se desea agregar funcionalidad a la plataforma (movimiento, activación por evento, etc.), se utilizaría un Blueprint que herede del Static Mesh Actor y ejecute la lógica correspondiente, como una animación de desplazamiento.

Escenario 2. Objeto recolectable

El jugador se desplaza por un escenario y entra en contacto con una moneda. La moneda debe desaparecer y aumentar el contador de puntos, sin detener el movimiento del personaje.

Analiza y responde:

- ¿Qué diferencia existe entre bloquear al personaje y detectar un contacto?
- ¿Qué tipo de interacción sería apropiada?
- ¿Qué condición debe cumplirse para sumar puntos?
- ¿Qué acción debería ejecutarse después de recoger la moneda?
- ¿Cómo se podría evitar que el mismo objeto se recoja varias veces?

Un sistema de colisiones detecta de forma innata cuándo una instancia de colisión entra en contacto con otra. A partir de esto, en Unreal Engine se pueden configurar eventos mediante los nodos **OnComponentBeginOverlap** (al comenzar el traslape) y **OnComponentEndOverlap** (al terminar el traslape). Sin embargo, esto no significa que las colisiones detengan el movimiento automáticamente como ocurriría en la vida real, para eso sería necesario configurarlas como *Block*.

Para recolectar una moneda, lo ideal —sin recurrir a un sistema de interacción por botón, como "Presionar E para recolectar"— es que **ninguno de los dos actores bloquee el movimiento del otro** y que, en el evento BeginOverlap del Blueprint de la moneda, se ejecute la lógica de recolección: La condición que debe cumplirse para sumar puntos es que el evento BeginOverlap se dispare, incremente la variable de puntaje, reproduzca (opcionalmente) un sonido y/o efecto visual, **e inmediatamente destruir el actor de la moneda**. Destruir el actor permite que la moneda no se recoja varias veces, ya que el evento BeginOverlap se puede disparar en múltiples fotogramas mientras las colisiones sigan traslapándose.

Conclusiones

A lo largo de esta investigación se comprendió que un videojuego 2D funcional no depende únicamente de sus recursos gráficos, sino de la correcta integración de sprites, cámara, física y colisiones. Los sprites dan forma visual a los objetos, pero son los componentes de colisión y la lógica condicional los que definen cómo interactúan entre sí. La cámara determina qué ve el jugador y cómo interpreta el espacio. Una cámara mal configurada puede volver injusto un nivel, mientras que una bien ajustada reduce la fricción y mejora la legibilidad.

La física y las colisiones, por otra parte, establecen las reglas de movimiento y contacto mediante respuestas como Block, Overlap e Ignore, permitiendo decidir qué objetos detienen, detectan o ignoran. En el caso de *Adventure Campus*, estas decisiones son importantes porque hay: plataformas que bloquean, monedas que detectan y se recolectan y objetos físicos con constraints.

Principalmente, la calidad de un juego 2D no está solo en lo que se ve, sino en cómo se comporta cada objeto bajo reglas claras y consistentes. La física, las cámaras y las colisiones son, en conjunto, el puente entre la intención del diseñador y la experiencia del jugador.

Bibliografía

Brown, M. (2023). *How to make a good 2D camera*. Game Industry Library. <https://gameindustrylibrary.com/documents/how-to-make-a-good-2d-camera-by-mark-brown>

Cornell University. (s.f.). *Game camera*. The Game Design Initiative at Cornell University. <https://www.cs.cornell.edu>

Epic Games. (2014). *Paper 2D components*. <https://dev.epicgames.com/documentation/en-us/unreal-engine/paper-2d-components>

Epic Games. (2021). *Camera*. Unreal Engine Documentation. <https://dev.epicgames.com/documentation/unreal-engine/camera>

Epic Games. (2024). *Orthographic camera in Unreal Engine*. <https://dev.epicgames.com/documentation/unreal-engine/orthographic-camera-in-unreal-engine>

Epic Games. (2025). *Collision in Unreal Engine*. <https://dev.epicgames.com/documentation/unreal-engine/collision-in-unreal-engine---overview>

Epic Games. (2025). *Paper 2D overview in Unreal Engine*. <https://dev.epicgames.com/documentation/unreal-engine/paper-2d-overview-in-unreal-engine>

Epic Games. (2025). *Paper 2D physics in Unreal Engine*. <https://dev.epicgames.com/documentation/unreal-engine/paper-2d-physics-in-unreal-engine>

Genbeta. (2013, 8 de abril). *SFML 2: Sprites y texturas*. <https://www.genbeta.com/desarrollo/sfml-2-sprites-y-texturas>

Hobby Consolas. (2021, 14 de abril). *¿Qué son los sprites en los videojuegos? Aprende con Hobby Basics*. <https://www.hobbyconsolas.com/reportajes/son-sprites-videojuegos-aprende-hobby-basics-847679>

Peake, B. (2016). *Paper2D tutorial in UE4* (Versión 1.1). Worcester Polytechnic Institute. <http://web.cs.wpi.edu/~imgd4000/d16/projects/proj-quickstart/TutorialDoc.pdf>

Platzi. (2026, 27 de enero). *Implementación de Cámara Follow en Videojuegos 2D con Unity*. <https://platzi.com>

Radu. (2025). *Camera lead. Game Mechanics Optimizations*. <https://github.com/raduacg/game-mechanics-optimizations>

Strayspark Studio. (2026, 25 de marzo). *UE5 project architecture: Folder structure, naming conventions, and asset management*. <https://www.strayspark.studio/blog/ue5-project-architecture-naming-conventions>

Thorson, M. (s.f.). *Celeste & TowerFall physics*. Maddy Makes Games. https://www.maddymakesgames.com/articles/celeste_and_towerfall_physics/index.html

Unity Technologies. (2025). *2D game perspectives reference*. Unity Manual. <https://docs.unity3d.com/6000.1/Documentation/Manual/2d-game-perspective-reference.html>

Unreal Engine. (2014, 18 de abril). *Collision filtering in Unreal Engine 4*. <https://www.unrealengine.com/blog/collision-filtering>